Topic 23 — Prompting Types: Zero-Shot, One-Shot, Few-Shot

Full Stack Python + AI Roadmap • Phase 3: Advanced Prompt Engineering • 3 layers: New → Intermediate → Expert

🌱 Layer 1 — New to the Concept

The "shot" in these names means **one example** you give the model inside the prompt:

- **Zero-shot** = zero examples — you just ask.
- **One-shot** = one example — you show one, then ask.
- **Few-shot** = a few examples — you show several, then ask.

You're deciding **how many worked examples to include** before asking the model to do the real task.

Analogy — teaching someone to write review summaries: **zero-shot** "summarize this in one line" (they figure it out); **one-shot** "here's an example, now do this one"; **few-shot** "here are 3 examples, now do this one" (they see the pattern clearly). More examples = more pattern to copy = more consistent output, at the cost of a longer, pricier prompt.

👩‍💻 Layer 2 — Real Prompts for Each

Zero-shot — just the instruction

```
prompt = """Classify the sentiment as POSITIVE, NEGATIVE, or NEUTRAL.

Review: "The delivery was late but the product quality is excellent."
Sentiment:"""
```

Modern LLMs are strong zero-shot for common tasks — often all you need. Start here; add examples only if it underperforms.

One-shot — one example to lock in the format

```
prompt = """Classify the sentiment as POSITIVE, NEGATIVE, or NEUTRAL.

Review: "Absolutely love it, works perfectly!"
Sentiment: POSITIVE
```

```
Review: "The delivery was late but the product quality is excellent."
Sentiment:"""
```

The single example shows the model *exactly* what output shape you want (just the label). Great when the format matters more than the range of cases.

Few-shot — several examples showing pattern & edge cases

```
prompt = """Classify the sentiment as POSITIVE, NEGATIVE, or NEUTRAL.

Review: "Absolutely love it, works perfectly!"
Sentiment: POSITIVE

Review: "Complete waste of money, broke in a day."
Sentiment: NEGATIVE

Review: "It's okay, does the job, nothing special."
Sentiment: NEUTRAL

Review: "The delivery was late but the product quality is excellent."
Sentiment:"""
```

ℹ️ **In-context learning:** the model "learns" the task from examples in the prompt, without any retraining. The examples teach the pattern *and* cover the possible outputs, so the model generalizes.

🧠 Layer 3 — Expert (when to use which, how it works)

1. The decision framework

```
Is the task common & simple (translation, basic classification, summarizing)?
├── Yes → Zero-shot. Try it first; modern models are strong here.
└── No / specific format / model keeps getting it wrong:
    ├── Need to fix the OUTPUT FORMAT mainly → One-shot
    └── Need to teach a PATTERN / edge cases / custom labels → Few-shot
```

✅ **Pro rule:** start zero-shot, escalate to few-shot only when needed. More examples cost more tokens and don't always help — sometimes they bias the model.

2. In-context learning — what's actually happening

The model is **not** retrained or fine-tuned. The examples live only in the prompt for that one call. The model recognizes the pattern and continues it — next-token prediction guided by the demonstrations. Once the call ends, that "learning" is gone. Hence *in-context* learning (from the prompt context, not weight updates).

3. Few-shot with the messages API — the proper structure

```
messages = [
    {"role": "system", "content": "Classify sentiment as POSITIVE, NEGATIVE, or NEUTRAL."},
    # example 1
    {"role": "user", "content": "Absolutely love it, works perfectly!"},
    {"role": "assistant", "content": "POSITIVE"},
    # example 2
    {"role": "user", "content": "Complete waste of money."},
    {"role": "assistant", "content": "NEGATIVE"},
    # the real request
    {"role": "user", "content": "The delivery was late but quality is excellent."},
]
```

Presenting examples as prior user/assistant turns is the idiomatic, most effective way to few-shot with chat models. LangChain's `FewShotPromptTemplate` automates exactly this.

4. Example selection matters (a lot)

- **Diversity** — cover the different cases/labels (don't give 3 positive examples then expect good negative handling).
- **Representativeness** — examples should resemble real inputs.
- **Order** — models sometimes weight the last example more (recency).
- **Consistency** — all examples must follow the exact output format you want (inconsistency confuses the model).

In advanced systems, examples are **retrieved dynamically** — pick the most similar examples to the current input from a bank ("dynamic few-shot," tying into your vector-DB/RAG knowledge).

5. The cost/reliability tradeoff

Type	Tokens/cost	Reliability	Best for
Zero-shot	Lowest	Good for common tasks	Simple, well-known tasks
One-shot	Low	Better format control	Fixing output shape
Few-shot	Higher	Most consistent	Custom/nuanced tasks, edge cases

Each example added is more input tokens on *every* call — at scale, few-shot has a real cost. Buy reliability with tokens only when you need it.

6. When few-shot isn't the answer

If you need many examples (dozens+) or deep domain knowledge, **fine-tuning** (training the model on your data) may beat few-shot — it bakes the pattern into the weights so you don't pay for examples every call. Rule of thumb: few-shot for a handful; fine-tune when you have many and call it constantly.

🎯 Interview Q&A Cards

Q: What do zero-, one-, and few-shot mean?

A: How many examples you include in the prompt before the real task — zero, one, or several. It's in-context learning: the model picks up the task from the prompt examples, not from retraining.

Q: When would you use few-shot over zero-shot?

A: When the task is nuanced, needs a specific output format or custom labels, involves edge cases, or the model underperforms zero-shot. Start zero-shot and escalate only when needed, since examples add token cost.

Q: How do you structure few-shot examples for chat models?

A: As alternating user/assistant turns representing each example, with the real query as the final user turn. The model treats the prior turns as demonstrations of how to respond.

Q: Few-shot vs fine-tuning?

A: Few-shot puts a handful of examples in the prompt (no training, but token cost each call). Fine-tuning trains the model on many examples, baking the pattern into the weights — better when you have lots of examples and call it constantly.

🪝 **Memory hook:** *"Shot" = one example. Zero = just ask; one = show one (fixes format); few = show several (teaches pattern). It's in-context learning (prompt only, no retraining). Start zero-shot, escalate as needed. Structure examples as user/assistant turns. Many examples + frequent calls → consider fine-tuning.*